SupplySense

Project map and run guide

A practical guide to the active code, database, demand model, local startup, and GitHub upload.

WHAT RUNS TOGETHER

1. Browser	2. Customer client	3. API	4. Database
Customer or manager	React + Vite localhost:5173	Express + Prisma localhost:4000	PostgreSQL 5433 native / 5432 Docker

Request path

The Vite client forwards **/api** requests to the Express API. The API reads and writes PostgreSQL through Prisma. Product photos are referenced by the client/server photo manifests.

Demand prediction path

The API rebuilds a 28-day sales series per warehouse and SKU, then calls **ml/predict_demand.py**. The script loads **ml/artifacts/supplysense_best_model.joblib** and returns forecasts for SKUs with at least seven days of recorded sales.

Choose a run method

NATIVE - COMPLETE FEATURES	DOCKER - CORE APP
Recommended for local development and manager demand forecasts. Uses Node.js, PostgreSQL, and Python with joblib.	Starts the client, API, and PostgreSQL. The current API image does not bundle Python or ml/, so demand prediction needs the native setup.

Checkout uses the local mock payment adapter. It does not charge a real card.

Repository map

This is a monorepo. The root npm workspaces are client and server; similarly named legacy folders are separate prototypes.

PATH	WHAT IT CONTAINS
client/	ACTIVE FRONTEND. React 19 and Vite. Main files: src/main.jsx mounts the app; src/App.jsx contains the customer, manager, cart, checkout, and order screens; src/styles.css contains layout and interaction styles. src/product-photo-manifest.json maps product names to image URLs; public/ holds static assets.
server/	ACTIVE BACKEND. Node.js and Express API. src/server.js starts the service; src/app.js configures middleware and routes; src/routes/ maps HTTP endpoints; src/services/ holds commerce, inventory, order, payment, supplier, and intelligence logic; src/config/, middleware/, lib/, and utils/ provide supporting code.
server/prisma/	DATABASE DEFINITION. schema.prisma defines PostgreSQL tables and relations; migrations/ contains versioned SQL; seed.js loads demo users, catalog, warehouses, inventory, and suppliers.
server/scripts/	DATABASE SUPPORT. seed-demo-sales-history.js adds local demo SALE/RETURN movements for demand prediction; refresh-product-options.js updates product option data.
ml/	PYTHON DEMAND PREDICTION. predict_demand.py is called by the API. requirements.txt lists joblib. model_script.ipynb is the notebook. artifacts/supplysense_best_model.joblib is the selected live bundle; other CSV and joblib files are analysis/training artifacts.
docs/	DESIGN AND API DOCUMENTATION. Requirements, architecture, database design, business rules, OpenAPI, security, and phase-by-phase implementation notes.
scripts/	LOCAL OPERATIONS. local-postgres.sh initializes/starts/stops PostgreSQL under .local/ on port 5433; verify-local.mjs checks a running local setup.
backend/	LEGACY PROTOTYPE. A separate Express + SQLite app on port 5000. It is not in the root npm workspaces and is not used by the current client/server app.
frontend/	LEGACY STATIC HTML PROTOTYPE. It is not the active React/Vite customer frontend. Use client/ for the current website.
docker-compose.yml	Optional local containers for PostgreSQL, API, client, and optional Redis. Docker's PostgreSQL uses host port 5432.
package.json	Root npm workspace and shortcuts. dev starts both active workspaces; db:* commands operate the server database; docker:* wraps Docker Compose.
.env.example	Safe configuration template. Copy it to .env for native development. The actual .env is ignored by Git.
output/pdf/	Generated project/run guide PDF. This folder is included in the repository so GitHub readers can download it.

Generated/local-only folders

node_modules/, client/dist/, .local/, .venv/, and tmp/ are generated or machine-local data and are ignored by Git. Do not commit local database files or temporary render previews.

Run the complete project locally

Use this path when you need the customer/manager application, PostgreSQL, and Python demand prediction together.

Prerequisites

Install Node.js 22 or newer, npm, PostgreSQL 16 command-line tools (initdb, pg_ctl, psql), and Python 3. The local PostgreSQL helper uses port 5433. On macOS, install PostgreSQL with your preferred package manager and ensure its bin directory is on PATH.

First-time setup

```
cp .env.example .env
npm install

python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r ml/requirements.txt

npm run db:local:start
npm run db:generate
npm run db:deploy
npm run db:seed
npm run dev
```

Open the app

Customer/manager website	http://localhost:5173
API	http://localhost:4000/api/v1
API liveness	http://localhost:4000/api/v1/health/live
PostgreSQL	localhost:5433 / database: supplysense

Demo data and accounts

The database seed creates demo users and the follow-up sales-history script adds sample recent sales across warehouses. Demo login credentials are listed in README.md; use them only for local development. Keep the seeded history if you want the manager demand prediction to have sufficient inputs.

Stop the services

Press Ctrl+C in the terminal running npm run dev. Stop local PostgreSQL with **npm run db:local:stop**. To start again later, reactivate .venv, then run npm run db:local:start and npm run dev.

Demand model, Docker, and GitHub

Demand model behavior

The API reads SALE and RETURN inventory movements for the recent 28-day window. It calls the Python predictor for SKUs with at least seven observed sales days. The selected bundle is ml/artifacts/supplysense_best_model.joblib; the implementation is a recursive lag-7 baseline that repeats the recent weekly pattern. Exponential smoothing is kept as a fallback for explicitly selected SKU requests. The larger supply_sense_demand_model.joblib and training CSVs are separate research artifacts, not the selected live bundle.

If a prediction reports insufficient history

Run the local database seed after migrations, confirm demo sales history is present, and select a warehouse with recent SALE/RETURN movements. Install joblib into the active .venv. The predictor is not a standalone web service.

Docker option

```
docker compose up --build
# open http://localhost:5173
docker compose down
```

Docker uses ports 5173 (client), 4000 (API), and 5432 (PostgreSQL), with a named database volume. The current server Dockerfile copies server/ but does not install Python or copy ml/, so use the native setup for forecast runs. Avoid starting Docker and native web services at the same time because both use ports 5173 and 4000.

Prepare the folder for GitHub

This working folder is not currently initialized as a Git repository. .gitignore excludes .env, .local/, node_modules/, .venv/, tmp/, and build outputs; output/pdf/ and .env.example remain shareable. Before pushing, check that .env is not staged. Replace demo credentials and development secrets before deploying anywhere public.

```
git init -b main
git add .
git status
git commit -m "Initial SupplySense project"
git remote add origin https://github.com/ACCOUNT/REPOSITORY.git
git push -u origin main
```

Replace ACCOUNT/REPOSITORY with the URL of your GitHub repository. Do not force-add .env.

Quick fixes

API unavailable: make sure npm run dev is still running and ports 4000/5173 are free. Database connection refused: check npm run db:local:status and DATABASE_URL in .env. Model unavailable: activate .venv before starting the API and install ml/requirements.txt. Manager prediction empty: use seeded demo sales or record completed sales for that warehouse.